

TIGA: COMPILING GRAPH MESSAGE PASSING AT SCALE

Anonymous authors

Workshop manuscript draft

ABSTRACT

Graph message passing is a common computational pattern in learning and numerical simulation, but explicit graph construction and edge-wise tensor operations can consume more memory than the final result. We present TIGA, a just-in-time compiler that keeps graph relations and reduction semantics visible until traversal and execution are selected. Generated relations can be consumed by aggregation without storing a complete edge list or message array. For explicit graphs that exceed device memory, a paged runtime streams topology and source fields through host memory while retaining a bounded temporary device working set. The interface accepts ordinary PyTorch tensors, with path-specific automatic differentiation and native storage controls for offloaded execution. Experiments compare forward latency and allocation with matched Torch and PyTorch Geometric implementations, and demonstrate billion-edge forward execution on a memory-limited GPU. The results motivate relation-aware compilation while exposing the host-staging costs of capacity-oriented paging. Gradient checks establish supported derivative semantics separately from the forward-performance measurements.

1 INTRODUCTION

Message passing describes computation through entities, interactions, local functions and reductions. In machine learning, this pattern includes graph neural networks, point-cloud neighborhoods and attention; numerical simulation uses related structures for particles and mesh operators. The same high-level pattern can require very different execution strategies. A stored sparse graph needs indexed traversal, a radius relation can exploit spatial locality, and a dense relation need not have an explicit edge array at all.

Tensor-oriented implementations often build connectivity first, gather fields for every edge, compute messages, and finally reduce them. For edge count and feature width denoted by E and F , a materialized message array uses storage proportional to EF , even when its sole consumer produces NF output values for N destinations. Dynamic coordinates can make the cost of constructing these intermediates recur at every call. When the input graph itself exceeds device memory, fusion alone is insufficient: execution must also control which topology and field pages reside on the accelerator.

TIGA, short for *Target-Independent Graph Acceleration*, separates the meaning of a message-passing program from its traversal, hardware mapping and storage residence. Target independence describes this separation, not equal backend coverage: current executable routes target CPUs and NVIDIA GPUs. The central design choice is to retain relation and reducer information before lowering to tensor and device operations. This enables two complementary mechanisms: consuming generated neighbors directly, and streaming stored graphs with a bounded temporary working set.

This paper contributes a relation-aware compilation interface and lowering strategy, a destination-page execution contract for graphs exceeding device memory, and an evaluation of their forward latency, allocation and capacity. The results establish useful execution paths rather than a universal framework ranking. In particular, the evaluation does not isolate every compiler optimization or establish end-to-end training throughput. Differentiation and distributed execution are supported on specific paths, with their boundaries stated separately from the central performance claims.

Relation to prior systems. PyTorch Geometric exposes message passing to graph-learning applications (Fey & Lenssen, 2019; Fey et al., 2025). Graphiler retains graph-related dataflow to

eliminate redundant work and storage (Xie et al., 2022); FeatGraph composes sparse traversal and feature schedules (Hu et al., 2020); Seastar generates fused forward/backward kernels (Wu et al., 2021); and GALA composes graph-training transformations (Lenadora et al., 2025). Tiga likewise exploits message semantics, but additionally retains generated relation constructors and explicit storage residence in its execution model. These design differences are not comparative performance claims against those compilers. GraphIt separates algorithms from schedules (Zhang et al., 2018), while TACO and SparseTIR separate sparse iteration and storage choices (Kjolstad et al., 2017; Ye et al., 2023). GraphChi and X-Stream establish that partitioned streaming can make large graphs executable on a single machine (Kyrola et al., 2012; Roy et al., 2013). Tiga builds on these principles for differentiable numerical field programs; neither fusion nor offloading is claimed as a new idea in isolation.

2 RELATION-AWARE COMPILATION AND BOUNDED EXECUTION

2.1 THE PROGRAM RETAINS INTERACTIONS, NOT EDGE ARRAYS

A graph maps each edge to a source and a destination. A message function reads source, destination and edge fields; a reducer combines messages for each destination. The result is

$$y_i = \text{finalize} \left(\bigoplus_{e:d(e)=i} \text{lift} \left(f(x_{s(e)}, z_i, a_e; \theta) \right) \right).$$

The reducer declares an identity, state update and finalization. Duplicate edges are separate contributions; empty rows follow the reducer’s identity and finalization convention. An optional node function consumes the reduced value. The edge function determines what is computed, while the graph determines which interactions exist.

For example, this program sums source features over a coordinate-dependent radius relation. Positions and features are ordinary Torch tensors on the same device; rows denote destinations and columns denote sources.

```
import tiga as tg

class NeighborSum(tg.MessagePassing):
    reducer = tg.sum()
    def edge(self, src, dst, edge):
        return src.x

graph = tg.Graph.radius(positions, cutoff=0.1)
y = NeighborSum()(graph=graph, src={"x": x}, dst={})
```

The interface does not require an edge list, although a consumer may select an explicit representation. Dense and triangular graphs similarly describe an interaction domain. For causal attention, the destination supplies a query, the source supplies a key/value, and an online-softmax reducer combines score and value messages. Preserving the reducer state admits streaming evaluation without a global score matrix, following the IO-aware principle exemplified by FlashAttention (Dao et al., 2022).

2.2 LOWERING PRESERVES LEGALITY BEFORE CHOOSING A SCHEDULE

Tiga uses MLIR for its compiler representations (Lattner et al., 2021). Domain IR retains field roles, graph construction and message/reducer regions. Iter IR introduces relation-coordinate traversal. Kernel IR assigns tiles and reduction work to execution resources. Task/Storage IR describes dependencies, physical copies and ownership where the selected path needs them. Tensor IR carries numerical expressions alongside these layers; it is not a mandatory final stage.

Three properties constrain the transformations. First, field roles distinguish a source ID from an edge position, even when both index floating-point arrays. Second, reducer associativity permits regrouping and commutativity permits reordering; arbitrary Python functions do not automatically

satisfy either property. Third, shape, dtype, layout and topology guards determine whether an executable remains legal. Floating-point reduction changes require numerical tolerances even when real-arithmetic semantics agree. NVIDIA lowering emits serialized Triton IR for provider compilation (Tillet et al., 2019); native CPU paths lower through LLVM. A provider interface alone does not establish a working backend on another vendor’s hardware.

Generated relations. For supported low-dimensional Euclidean radius workloads, a cell directory limits traversal to neighboring cells. An exact distance predicate selects accepted sources, whose fields can be consumed immediately by the reduction. This avoids storing accepted-edge indices, distances and messages globally. A custom metric needs a compatible spatial bound before that traversal is legal.

For exact kNN, a destination program scans candidate tiles and keeps a padded top-k state, ordered by squared distance and source ID. If the smallest key in a candidate tile cannot improve the retained state, sorting and merging that tile are skipped; its distances are still computed. This is exact selection pruning, not approximate neighbor search or a subquadratic algorithm. Compatible new coordinates can rebind an existing executable without capture and lowering. Coordinates remain live inputs: executable reuse never implies reuse of previously selected neighbors.

2.3 PAGING SEPARATES CAPACITY FROM KERNEL SPEED

Explicit graphs larger than device memory use a native storage path. Topology and source fields remain in a persistent store. The runtime loads a bounded destination-row page, gathers its source fields on the host, stages the page to CUDA and runs its reduction. It assembles the final output in a resident device tensor. The capacity constraint is therefore

$$M_{\text{output}} + M_{\text{active topology}} + M_{\text{staged fields}} + M_{\text{temporary}} \leq B_{\text{device}},$$

where the budget applies to Tiga-managed device buffers, not other processes or the full CUDA context. The output must still fit the device. Page topology and field versions must agree; transfer must finish before consumption; and storage cannot be recycled until its consumers finish.

The current implementation gathers features in page-edge order, potentially repeating a source value many times. It also returns completed output pages through host bytes before writing the resident result. These choices bound temporary memory but can dominate runtime. The measured configuration is serialized, without prefetch. Optional read-ahead does not implement a complete disk–host–device overlap pipeline. Ordinary Torch tensors remain under Torch’s allocator; Tiga-managed paging uses native tensors rather than silently taking ownership of arbitrary Torch storage.

Differentiation boundary. Supported Torch-facing programs interoperate with Torch autograd. CUDA EdgeNN paths recompute local activations in a symbolic vector–Jacobian product (VJP); some other compiled forwards replay Torch operations during backward. CPU paging has a supported native VJP, but CUDA paged execution and the dense streaming attention path evaluated here have no compiled backward. Derivatives act on the selected relation, not discrete radius membership or kNN selection. Appendix B distinguishes the numerical checks from training performance evidence.

3 EVALUATION

Setup. The main host has a Ryzen 7 255, approximately 44 GiB RAM, NVMe storage and an RTX 5070 Ti with 16 GiB device memory. Software comprises Torch 2.11.0+cu128, Triton 3.6.0, PyG 2.8.0.post1, pyg-lib 0.9.0+pt211cu128 and driver 595.84. These are shared, non-clock-locked hosts. Timings are synchronized host wall time through completed output. The comparisons use ordinary Torch inputs; paging and distributed runs use native tensors. All performance results are FP32 forward evaluations, not training steps. Figure 1 shows matched shapes and semantics, with lower latency and allocation preferable.

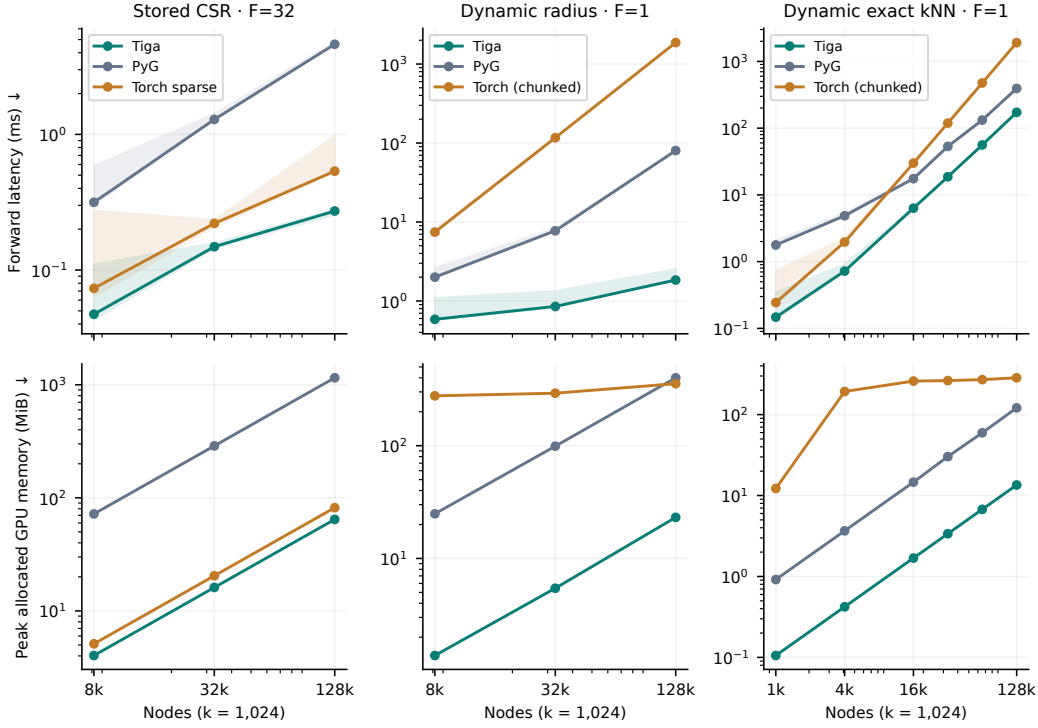


Figure 1: Matched forward latency and allocated GPU memory. Columns are distinct workloads: stored CSR has 32 features, dynamic radius and kNN have one. Both axes are logarithmic. Dynamic calls include rebuilding from changed coordinates; Torch (chunked) uses a pure Torch builder and aggregation.

3.1 GENERATED AND STORED RELATIONS

Stored CSR uses degree 32 and 32 features at 8,192–131,072 nodes. Baselines are Torch sparse multiplication and PyG COO gather–scatter. Dynamic radius uses a scalar distance-weighted sum, nominal degree 32 and the same node sizes. Dynamic kNN uses a scalar neighbor sum with $k = 16$ at 1,024–131,072 points. Each dynamic call changes coordinates, constructs the relation and consumes it; unchanged-input topology caching is not timed as construction.

PyG baselines use their respective radius/kNN builder plus COO message passing. Pure Torch baselines compute distances in bounded row chunks and then use CSR/sparse multiplication for radius, or top-k followed by gather/sum for kNN. The distance-block budget is 16,777,216 pairs. These eager all-pairs baselines do not represent the best possible spatial-index implementation in Torch. Complete explicit-reference/PyG edge sets agree for both coordinate snapshots. Radius inputs avoid ambiguous floating-point cutoffs, and the neighbor cap does not truncate the measured relations.

Each of 36 configurations runs in a fresh process, with provider order randomized, four warmups and ten timed calls checked at relative and absolute tolerance $3 \cdot 10^{-4}$. Curves show medians and sample ranges. Peak Torch-allocated bytes include inputs, output, graph representations and temporaries, but exclude allocator reserve, CUDA context and other processes. Largest-shape audits found no extra Tiga-native device-buffer allocation on these Torch paths. Compilation is outside the warm timing window.

Stored CSR achieves $1.49\text{--}1.97\times$ speedup and approximately $1.27\times$ lower allocation than Torch sparse across the three sizes. At 131,072 radius points, Tiga takes 1.84 ms versus 80.24 ms for PyG and 1,870.09 ms for chunked Torch, with allocation of 23.14, 400.22 and 355.01 MiB respectively. This comparison combines algorithmic traversal choices and materialization/fusion effects; it does not isolate the contribution of any one compiler pass.

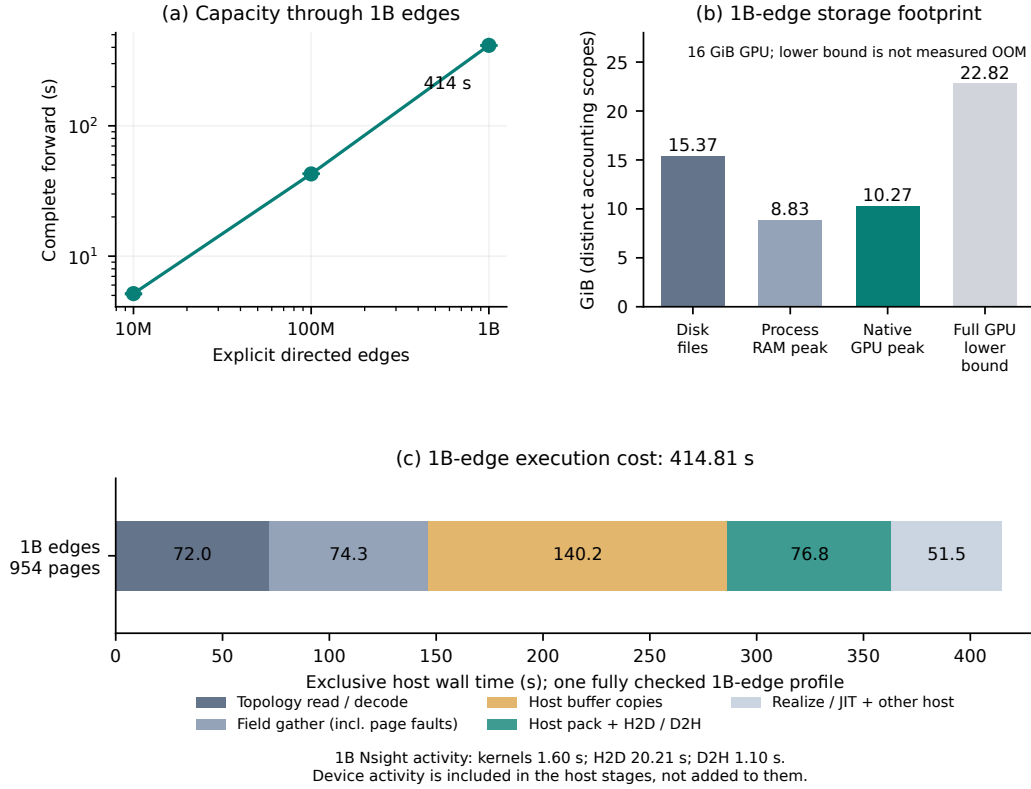


Figure 2: Capacity through 1B explicit edges. The full-residency requirement is calculated; other memory bars have distinct measured accounting scopes. The host-stage breakdown is one separately instrumented forward call, not extra time added to the three-trial capacity sweep.

At 131,072 kNN points, Tiga takes 172.37 ms versus 393.21 ms for PyG and 1,908.43 ms for chunked Torch; the allocations are 13.50, 121.13 and 285.00 MiB. The PyG-relative speedup ranges from $2.28\times$ to $12.06\times$ over six sizes. These results concern scalar messages on the tested geometries, not wide-feature EdgeNNs or arbitrary point distributions.

3.2 BILLION-EDGE CAPACITY AND ITS COST

The capacity workload explicitly stores a periodic directed CSR graph with 16 neighbors per destination and 32 features. The sweep contains 10M, 100M and 1B edges with three fresh-process calls per size. Ingest is excluded; forward includes loading, staging, compilation as encountered, computation and output assembly. All output values are checked after timing against a closed-form oracle; the execution still traverses every stored edge.

Pages contain 65,536 destination rows, the managed-device budget is 12 GiB, and prefetch is disabled. At 1B edges, the graph has 62.5M nodes. Int64 CSR, source and output together require at least 22.82 GiB for full residency (18.86 GiB with int32 CSR), exceeding device capacity. These are calculated lower bounds, not measured baseline OOMs.

The 1B median is 414.25 s (411.36–415.61), with maximum native GPU peak 10.27 GiB and forward process RSS 8.83 GiB. Disk topology/source storage is 15.37 GiB, and the resident output alone is 7.45 GiB. Sampled device-wide usage, including unrelated services and pools, reaches 13.25 GiB.

A separate checked 1B profile spends 414.81 s in its forward window: 71.98 s topology read/decode, 74.27 s field gathering, 140.22 s host-buffer copying, 76.79 s packing/transfers and 51.55 s setup/assembly/other work. Nsight records 1.60 s of CUDA kernels and 20.21 s H2D activity inside

those host intervals. Source expansion turns 8 GB of stored features into 128 GB of gathered values. Host staging, not reduction arithmetic, dominates. The result establishes execution beyond device memory, not optimal offload performance, capacity beyond host RAM, or a maximum supported edge count. A hand-chunked peer could also trade storage traffic for capacity.

Distributed scope. For a separate two-host spatial-mesh experiment, a 5070 Ti and 4070 Ti SUPER each own half the destination rows and exchange a one-layer halo through NCCL Socket transport before local computation. At 22.51M directed edges, paired-rank latency is 44.22 ms, versus 23.00 ms on the faster GPU alone. Thus the current fixed partition does not establish a distributed speedup, despite only 1.125 MiB of halo traffic. Appendix C gives the timing conditions and interpretation.

4 DISCUSSION AND CONCLUSION

Relation-aware compilation can reduce work and storage that arise from prematurely expanding interactions. The performance measurements support this opportunity for selected generated-relation workloads. Paging addresses a different constraint: it makes an explicit billion-edge computation executable on a memory-limited GPU, while exposing substantial host-staging overhead. These mechanisms benefit from a shared separation of relation semantics, traversal and storage, but neither makes every graph program efficient.

The present evidence leaves three important questions open. A same-search materialized-versus-fused ablation is needed to separate traversal gains from intermediate elimination. Real-data, wide-feature applications and complete forward/backward steps are needed to establish learning-workload benefits. Finally, first-call compilation costs and amortization need to be reported alongside warm execution. Capacity comparisons additionally need a matched bounded-memory baseline before claiming an advantage over manual chunking. These missing measurements limit attribution and generalization; the reported curves do not substitute for them.

The implementation also has path-dependent differentiation and backend coverage. CUDA paging remains forward-only, output storage must fit the device, and distributed ownership is fixed. The current results motivate improving source staging and output assembly, and assessing load-aware partitioning rather than assuming that a small halo guarantees strong scaling. Tiga provides a concrete substrate for exploring these choices without changing the local message and reducer interface.

REPRODUCIBILITY STATEMENT

Section 3 and Appendix A specify workloads, timing boundaries, validation and allocation accounting. Figures are generated from stored execution records with source/compiler hashes and configuration checks; they are not predictions. The numerical examples distinguish compiled forward execution from semantic gradient replay. No new measurement is inferred from replotting an existing record.

AI USE STATEMENT

AI coding and writing assistants were used in software implementation and testing, experiment design and tooling, literature discovery, interpretation of measurements, figure/code preparation, and manuscript drafting and revision. The numerical results reported here originate from recorded program executions, not generated estimates. Automated numerical and provenance checks were used to validate outputs, gradients on supported paths, and the recorded evidence. The human author is responsible for the final content, attribution and claims.

REFERENCES

- Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 16344–16359, 2022. doi: 10.52202/068431-1189. <https://proceedings.neurips.cc/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html>.
- Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. <https://arxiv.org/abs/1903.02428>.
- Matthias Fey, Jinu Sunil, Akihiro Nitta, Rishi Puri, Manan Shah, Blaž Stojanovič, Ramona Bendias, Alexandria Barghi, Vid Kocijan, Zecheng Zhang, Xinwei He, Jan Eric Lenssen, and Jure Leskovec. PyG 2.0: Scalable learning on real world graphs. In *Temporal Graph Learning Workshop at KDD*, 2025. <https://arxiv.org/abs/2507.16991>.
- Yuwei Hu, Zihao Ye, Minjie Wang, Jiali Yu, Da Zheng, Mu Li, Zheng Zhang, Zhiru Zhang, and Yida Wang. FeatGraph: A flexible and efficient backend for graph neural network systems. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2020. <https://arxiv.org/abs/2008.11359>.
- Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. The tensor algebra compiler. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA): 77:1–77:29, 2017. doi: 10.1145/3133901. <https://tensor-compiler.org/files/kjolstad-oopsla17-tensor-compiler.pdf>.
- Aapo Kyrola, Guy Blelloch, and Carlos Guestrin. GraphChi: Large-scale graph computation on just a PC. In *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 31–46, 2012. <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/kyrola>.
- Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 2–14, 2021. doi: 10.1109/CGO51591.2021.9370308. <https://doi.org/10.1109/CGO51591.2021.9370308>.
- Damitha Lenadora, Nikhil Jayakumar, Chamika Sudusinghe, and Charith Mendis. GALA: A high performance graph neural network acceleration language and compiler. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2), 2025. doi: 10.1145/3763113. <https://charithmendis.com/assets/pdf/25-oopsla-gala.pdf>.

- Amitabha Roy, Ivo Mihailovic, and Willy Zwaenepoel. X-Stream: Edge-centric graph processing using streaming partitions. In *ACM SOSP*, 2013. <https://sigops.org/s/conferences/sosp/2013/papers/p472-roy.pdf>.
- Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019. doi: 10.1145/3315508.3329973. <https://www.eecs.harvard.edu/~htk/publication/2019-mapl-tillet-kung-cox.pdf>.
- Yidi Wu, Kaihao Ma, Zhenkun Cai, Tatiana Jin, Boyang Li, Chenguang Zheng, James Cheng, and Fan Yu. Seastar: Vertex-centric programming for graph neural networks. In *EuroSys*, 2021. doi: 10.1145/3447786.3456247. https://www.cse.cuhk.edu.hk/~jcheng/papers/seastar_eurosys21.pdf.
- Zhiqiang Xie, Minjie Wang, Zihao Ye, Zheng Zhang, and Rui Fan. Graphiler: Optimizing graph neural networks with message passing data flow graph. In *Proceedings of Machine Learning and Systems*, volume 4, 2022. https://proceedings.mlsys.org/paper_files/paper/2022/hash/a1126573153ad7e9f44ba80e99316482-Abstract.html.
- Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. SparseTIR: Composable abstractions for sparse compilation in deep learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023. doi: 10.1145/3582016.3582047. <https://arxiv.org/abs/2207.04606>.
- Yunming Zhang, Mengjiao Yang, Riyadh Baghdadi, Shoaib Kamil, Julian Shun, and Saman Amarasinghe. GraphIt: A high-performance DSL for graph analytics. *Proceedings of the ACM on Programming Languages*, 2(OOPSLA), 2018. <https://arxiv.org/abs/1805.00923>.

A EXPERIMENTAL PROTOCOL AND INTERPRETATION

Stored and dynamic comparisons. Stored CSR excludes relation construction from timing. PyG’s COO representation is included in its allocation; Torch sparse provides a peer that does not materialize COO messages. Radius nominal degree is 32, with a cap of 128 verified not to truncate either coordinate snapshot. kNN excludes self-edges and uses direct squared differences in the reference, avoiding a matrix-product distance identity at close separations. Every timed output is checked outside the measurement window. Sample ranges describe variation within the retained calls, not cross-process confidence intervals. The dynamically generated cases have one feature; the stored CSR and capacity cases have 32.

Capacity accounting. Topology and source files occupy 16.50 GB (15.37 GiB). The 1B graph’s two-billion-scalar output is checked exactly after each retained run. Native device counters do not include unrelated allocations; process RSS is not total machine RAM consumption; and disk footprint is not a simultaneous resident allocation. The profiled outer call takes 417.05 s including profiler start/stop, whereas the inner window takes 414.81 s. Exclusive host intervals sum to the inner window. Device kernel/copy activity lies within these intervals and must not be added again. mmap gathering includes page-fault IO, so it is not a pure disk-service timer. The graph fits host RAM. Measured H2D traffic is 144.50 GB and D2H traffic is 8 GB; repeated source gathers explain why transfer volume exceeds the stored input size.

B DIFFERENTIATION CHECKS

The EdgeNN check constructs a radius graph on 128 three-dimensional positions. Each message concatenates source features of width eight and the displacement from destination to source, applies an 11–16–4 ReLU MLP, and sums at the destination. The loss is the mean squared output. An explicit Torch gather, the same MLP parameters and destination `index_add` provide a reference. Output, source-feature, position and parameter gradients are compared; the position derivative acts on displacements for the selected edges, not radius membership. Using the same resolved graph isolates message and gradient semantics rather than independently validating the neighbor search.

The attention check uses the exact triangular relation, two heads of width 64, and an online-softmax reducer. CUDA streaming output is compared with explicit FP32 Torch scores, causal masking, softmax and value multiplication. A separate `program.reference` call checks query/key/value gradients against independent Torch leaf tensors under a random output cotangent. This reference materializes edges and messages. It is not a streaming backward kernel. Checks cover sequence lengths 1, 7 and 65 on CPU, and 1, 7, 65 and 128 on CUDA, including singleton and partial-tile cases. These tests establish first-order agreement, not training throughput or universal higher-order differentiation.

C DISTRIBUTED NEGATIVE RESULT

Nonperiodic hexahedral meshes of side lengths 32, 64 and 96 connect nodes sharing an element in both directions without self-edges: 797,816, 6,596,856 and 22,508,920 directed edges. A 16-feature neighbor sum is evaluated, not a complete FEM solve. Equal destination partitions exchange one interface layer across a spatial plane; the boundary-row fraction falls from 6.25% to 2.083%.

Both hosts run matching compiler/runtime sources and NCCL 2.28.9 Socket transport. Two warmups precede five changed-input calls; the median is taken over paired-rank maximum durations. Timing includes exchange, scheduling, computation and synchronization, but excludes topology setup, the control barrier and checking. Replicated topology setup does not demonstrate distributed topology capacity beyond one GPU.

At the largest size, full-graph medians are 89.75 ms and 23.00 ms on the two cards, versus 44.22 ms distributed. Existing services occupy roughly 3 GiB on the main host and 12 GiB on the Ubuntu/WSL second host. Hardware, environment and contention are not independently controlled; the gap cannot be attributed solely to GPU compute throughput. This is a deployment-specific result, not a universal limit of graph partitioning.